

Examen Parcial de IA

(9 de Abril de 2026)

Duración: 110 min.

1. (4 puntos) Debido al continuo colapso de los juzgados y las continuas quejas de los funcionarios de la Administración de Justicia exigiendo mejoras salariales, el gobierno nos pide que ayudemos con la reestructuración del sistema de justicia Español. El Estado quiere mejorar la eficiencia del sistema con un nuevo procedimiento de asignación de plazas de funcionarios a los juzgados. Cada funcionario $f_i \in \{f_1 \dots f_F\}$ tiene un rol (juez, fiscal, abogado defensor, administrativo...), acepta un cierto sueldo mínimo (Sf_i) y acepta una distancia máxima a recorrer entre su casa y su trabajo (Df_i). Cada juzgado $j_k \in \{j_1 \dots j_J\}$ dispone de x_r^k plazas nuevas por cada rol r que es diferente para cada juzgado j_k . El ministerio paga cada plaza del rol r a un precio máximo determinado (S_r^k) que es diferente para cada juzgado j_k , y dispone de una tabla de distancias entre las viviendas de los funcionarios y los juzgados.

Se quiere asignar funcionarios a juzgados de forma que se maximice el ahorro económico respecto a lo que inicialmente está dispuesto a pagar el ministerio ($S_r^k - Sf_i$, y entendemos que si una plaza no se cubre **no** hay ahorro), se minimicen las distancias que los funcionarios deberán recorrer a su trabajo (para mejorar su rendimiento), y se cubra el máximo número de plazas nuevas, priorizando este último factor respecto a los dos anteriores. Supondremos que el número de funcionarios que solicitan plaza (F) es mucho mayor que el número total de plazas nuevas (P), pero las restricciones de rol, sueldo y distancia pueden dejar plazas nuevas sin asignar.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (estado o solución inicial, operadores, restricciones, función heurística,...). Para cada una de ellas se ha de comentar los diferentes elementos de la propuesta, analizando si la técnica escogida es adecuada para este problema, si cada uno sus elementos son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

- a) Queremos usar A^* , definiendo como representación del estado la asignación de funcionarios a plazas nuevas. El estado inicial es la asignación vacía. Como operadores tenemos en primer lugar el de asignar un funcionario a una plaza (si su rol coincide y si el sueldo y la distancia son convenientes). El coste de este operador es Sf_i (el sueldo mínimo del funcionario asignado). También tenemos un operador de desasignar un profesional de una plaza, cuyo coste es $-Sf_i$. La función heurística h que se pretende usar es el sumatorio de los sueldos máximos determinados (S_r^k) para las plazas que quedan por asignar.

El algoritmo A^* es una pésima opción para este problema por diversos motivos. El más importante es que en este problema, hasta la asignación nula es solución (si bien terrible), con lo cual el algoritmo termina inmediatamente/no termina/no sabe cuando terminar: el tener la posibilidad de no-asignaciones hace que sea difícil adaptar este algoritmo. Además, estamos en un caso en que se pide optimizar un conjunto de criterios, lo cual hace que el óptimo no esté claramente definido. Esta técnica tiene una complejidad demasiado elevada. Además, es poco probable encontrar una heurística admisible.

Obviando lo anterior, el estado inicial es correcto, partir de una asignación vacía e ir añadiendo sería correcto por lo general para A^* .

El operador asignar es correcto ya que nos permite ir construyendo la solución y realiza todas las comprobaciones pertinentes (rol adecuado, distancia adecuada, sueldo no supera máximo). Su factor de ramificación sería $O(F * P)$. Pero su coste es incompleto, ya que solo considera el sueldo del funcionario asignado, como veremos más adelante.

El operador de desasignar es directamente incorrecto para el algoritmo A^* y no tiene sentido. Además, su coste es negativo, lo cual es imposible para un A^* .

El heurístico h no es admisible por varias razones: la primera es que el momento en el que $h = 0$ no coincide con el momento en el que se encuentre un supuesto óptimo (la función h nos fuerza a asignar todas las plazas nuevas, pero la solución óptima puede requerir no asignarlas todas, por lo que podemos llegar al óptimo y que la función h nos diga que aun no lo hemos alcanzado); la segunda es que h está estimando el precio a pagar a partir del precio máximo de la plaza, que puede

ser mayor al que finalmente se le pague al funcionario (y por lo tanto el heurístico es pesimista, nos dice que hemos de pagar más de lo que en realidad se deberá pagar)

La función g es el sumatorio de los sueldos de los funcionarios asignados. la función h es una estimación (pesimista) de los sueldos que quedan por asignar. Aunque a priori $g + h$ está alineado con sus sumandos en las mismas unidades, esta propuesta de solución de A^* no está teniendo en cuenta dos criterios más a maximizar/minimizar: la distancia entre el domicilio de los funcionarios y su plaza asignada (a minimizar, se podría añadir en el coste del operador con un factor de ponderación) y el número de plazas cubiertas (a maximizar, esto es casi imposible hacerlo en A^* ya que el algoritmo siempre minimiza $g + h$ y es difícil establecer en este caso el momento preciso en el que $h = 0$).

Hay dos puntos extra en este apartado. En primer lugar, las plazas de un juzgado, para un mismo rol, son indistinguibles. Por tanto, se podría cambiar el operador de asignar de plaza a juzgado (ramificación baja a $O(F \cdot J)$) sin mucho problema, aunque debemos añadir aplicabilidad para no pasarnos del número de plazas de un rol x_r en ese juzgado. Por otro lado, y con mayor complejidad general, se podría conseguir un A^* funcional mediante considerar para cada funcionario o plaza la decisión de qué se hace con ella (asignar algo o no asignar nada). De esta forma, podríamos tener como condición de parada el haber tomado una decisión para todo, y a partir de aquí podríamos tal vez empezar a arreglar el resto de problemas como la admisibilidad o los costes. Sin algo como esto, todo este apartado no tiene sentido.

- b) Queremos usar optimización de restricciones. Como variables, en primer lugar, tenemos las *plazas* por cubrir (una por cada plaza nueva), y sus dominios son el conjunto de funcionarios f_i que tienen el rol adecuado para dicha plaza. También tenemos un conjunto de variables *precio_plaza* (una por plaza) cuyo dominio es el precio en euros al que se paga el sueldo de dicha plaza. También tenemos otro conjunto de variables *distancia_plaza* (una para cada plaza) con dominio el listado de distancias entre el edificio del juzgado de la plaza y la vivienda de los funcionarios listados en la variable *plaza* (y en el mismo orden). Por último, tenemos una variable '*objetivo*' con valor numérico arbitrario (pero positivo). Como restricciones, tenemos una restricción n-aria que mira que todas las plazas tengan exactamente un funcionario asignado, un conjunto de restricciones binarias entre cada variable de plaza y su correspondiente *precio_plaza* (número de restricciones igual al número de plazas) que indica que la variable de *precio_plaza* debe ser más grande que el sueldo mínimo del funcionario asignado a la variable plaza. Tenemos una restricción binaria similar para cada par plaza y *distancia_plaza*, donde la última toma como valor la distancia entre la vivienda del funcionario asignado y el juzgado de la plaza seleccionada. Tenemos una restricción n-aria que junta todas las variables de *distancia_plaza*, *precio_plaza* y la variable *objetivo* haciendo que la última tome como valor la suma de todas las variables de distancia más la suma de todos los sueldos multiplicada por una constante positiva λ . Finalmente, marcamos al algoritmo que el problema es un problema de minimización.

Usar optimización de restricciones puede ser una buena idea, ya que nos permite optimizar un criterio hasta llegar a una cantidad óptima. Si bien el coste de llegar al óptimo puede ser demasiado, el algoritmo saca soluciones subóptimas de vez en cuando. Debería ser aceptable siempre y cuando se pueda modelar de forma adecuada.

En el caso de la representación escogida, una asignación de valores a variables incluye qué funcionario va a cada plaza, lo cual es correcto. Sin embargo, tal como está ahora mismo, se exige que cada plaza tenga un funcionario, haciendo que la primera restricción propuesta no tenga sentido ya de entrada. Igual que en el apartado a) esto significa que no exploramos todas las posibles soluciones. Podemos arreglarlo añadiendo un valor 'no-asignado' a cada dominio de plaza. La representación incluye variables extra (*precio_plaza*, *distancia_plaza* y *objetivo*) para poder minimizar o maximizar los criterios del problema.

La primera restricción (R1) se plantea como una restricción n-aria que cubre todas las variables plaza para comprobar que cada una tenga exactamente un funcionario asignado. Como ya se ha dicho antes, por el enunciado esto no tiene sentido, ya que puede que se dejen plazas sin asignar, y además es una restricción que no tiene sentido incluir ya que la representación escogida ya nos asegura esta propiedad.

La segunda restricción (R2) se plantea como una restricción binaria entre las variables plaza y su correspondiente *precio_plaza* que comprueba que el valor asignado a *precio_plaza* es mayor que

el sueldo mínimo del funcionario asignado a la variable plaza. Esta restricción es correcta y tiene sentido.

La tercera restricción (R3) se plantea como una restricción binaria entre las variables plaza y distancia_plaza, donde la última toma como valor la distancia entre la vivienda del funcionario asignado y el juzgado de la plaza seleccionada. Es una restricción que le facilita al solver el acceso de los valores distancia haciéndolas valores explícitos de una variable asignada, pero en solvers modernos habría formas de obtener la distancia a partir de la asignación de funcionario en la variable plaza.

La última restricción (R4) se plantea como una restricción n-aria que cubre las variables precio_plaza, distancia_plaza y objetivo para computar, en la variable objetivo, el valor de una función a minimizar (la suma de todas las variables de distancia más la suma de todos los sueldos multiplicada por una constante λ para equilibrar la influencia de cada sumatorio). Esta restricción n-aria no se puede convertir en restricciones binarias, pero esto no es un problema para un solver moderno. Faltaría añadir otro criterio en la fórmula que computa el valor de la variable objetivo: dado que podemos dejar plazas sin asignar pero nos piden maximizar el número de plazas nuevas asignadas y este es, de hecho el criterio principal a maximizar, deberíamos añadir el número de plazas asignadas como un elemento más de la suma ponderada de forma que minimizar la variable objetivo maximice el número de plazas asignadas, o girar la fórmula de forma que maximizar la variable objetivo minimice y maximice los criterios de forma adecuada.

Falta una restricción para asegurar que un funcionario no esté asignado a dos plazas diferentes. Esta restricción se puede tener como $P*(P-1)/2$ restricciones binarias.

No se menciona en el enunciado que se tenga en cuenta la restricción de máxima distancia. Esto se puede conseguir con un preprocesado rápido de los dominios de las variables plaza, o con una restricción binaria extra entre cada par de variables plaza y distancia_plaza.

También queda un tanto ambiguo si las variables precio_plaza permiten pagar una plaza por encima de lo establecido (S_r^k), esto se debería tener en cuenta en su dominio, o de nuevo preprocesando los funcionarios por cada plaza.

De entre todas las posibilidades propuestas para este problema, esta es la que tiene más potencial de funcionar sin solo proponer soluciones triviales.

2. (6 puntos) Una de las labores de los equipos de comunicaciones en una red es distribuir el flujo de paquetes que le llegan a un equipo entre los diferentes equipos con los que está conectado, optimizando la calidad de la distribución de paquetes de forma local.

Una posible estrategia de distribución de paquetes sería decidir a priori cuantos de los paquetes que se reciben en un cierto segundo se envían por cada conexión, sin preocuparnos exactamente de a donde deben ir cada uno.

Un equipo de comunicaciones recibe un cierto número de paquetes por segundo (P) que tiene que distribuir entre sus conexiones de salida. Para cada una de las N conexiones de salida del equipo se conocen tres informaciones: su capacidad (en número de paquetes por segundo), el tiempo de retraso que introduce la conexión (tiempo medio adicional que añade el nodo de salida al tiempo de llegada a destino de cada paquete) y el número medio de saltos que hará cada paquete hasta llegar a su destino.

Deseamos calcular el número de paquetes que debemos enviar por cada conexión de salida para optimizar la distribución de paquetes de manera que se minimice el retraso y el número medio de saltos de los paquetes. Evidentemente se han de enviar todos los paquetes que llegan.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (estado o solución inicial, operadores, función heurística,...). Para cada una de ellas se ha de comentar los diferentes elementos de la propuesta, analizando si la técnica escogida es adecuada para este problema, si cada uno de sus elementos son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

- a) Se plantea solucionarlo mediante Hill-Climbing, usando como solución inicial el repartir todos los paquetes a partes iguales entre todas las conexiones de salida del equipo. Como operador utilizamos mover cierta cantidad de paquetes de una conexión a otra. Como función heurística usamos el sumatorio, para todas las conexiones con paquetes asignados, del producto entre el retraso de la conexión y el número de saltos.

Hill-Climbing es, a priori, una técnica adecuada para este problema en que se nos pide minimizar según unos criterios pero no se nos exige un óptimo global.

La solución es válida (es una asignación completa de paquetes a conexiones) siempre que el número medio de paquetes sea menor o igual que la capacidad de la conexión que admita menos paquetes, en caso contrario podría darnos una no-solución (si hay alguna conexión cuya capacidad es menor que P/N , asignaríamos a esa conexión un número de paquetes por encima de su capacidad). El coste de calcular la solución es $O(N)$, ya que no nos están pidiendo que asignemos paquetes concretos a conexiones, sino que nos piden calcular cuantos paquetes han de ir por cada conexión antes de que lleguen los paquetes, y para hacer esto solo hay que hacer el cálculo P/N , quedarnos con la parte entera de ese número e ir asignando ese número a cada conexión (en la última conexión o en la primera hay que ajustar el número para que el total sean P paquetes). Lo que podemos decir sobre la calidad de la solución inicial es que equilibra el retraso de todas las conexiones, no será la mejor solución inicial (porque no tiene en cuenta el número de saltos), pero tampoco será la peor.

El operador *mover* nos permite generar todas las soluciones posibles, ya que todos los paquetes están asignados desde la solución inicial y podemos moverlos de 1 en 1 o de n en n desde cualquier conexión origen a cualquier conexión destino. Como empezamos con una solución inicial que asigna los P paquetes entre las conexiones, con este operador no podemos generar un tipo de no-solución (asignar menos de P paquetes o más de P paquetes), pero debería comprobar las restricciones del problema (no superar los límites de capacidad de las conexiones de destino, y asegurarse que hay suficientes paquetes en la conexión de origen) para que no salga del espacio de soluciones. Al poder mover paquetes de cualquier conexión a cualquier otra y tener un parámetro extra para poder mover más o menos paquetes con el mismo operador el factor de ramificación sería $O(N^2 \times P)$ (para cada conexión origen del movimiento se puede escoger entre todas las otras conexiones como destino del movimiento, y se escoge el número de paquetes que se mueven), lo que supone un factor muy grande.

Es importante remarcar que en este problema el operador de mover nos da cobertura total de todo el espacio de soluciones y que no tiene sentido añadir un operador de intercambio / swap para evitar el bloqueo del operador mover cuando las conexiones están muy llenas, por varios motivos:

- en este problema todos los paquetes tienen el mismo tamaño, no tenemos paquetes más grandes y paquetes más pequeños. Si en la conexión de destino hay capacidad para un paquete más, lo podemos mover, y si no cabe no se mueve.
- en este problema no estamos asignando paquetes concretos, solo números de paquetes (no estamos distinguiendo los paquetes individuales). Por lo tanto un operador que intercambie un número de paquetes x de la conexión c_i con x paquetes de la conexión C_j resulta en la misma solución (las dos conexiones tienen el mismo número de paquetes que antes; un operador que intercambie un número de paquetes x de C_i con un número de paquetes y de C_j (siendo $x > y$) es equivalente a mover $x - y$ paquetes de c_i a c_j (restamos $x - y$ al número de paquetes de c_i , añadimos $x - y$ al número de paquetes de c_j).

La función heurística es aparentemente correcta ya que al minimizarla llegamos a soluciones mejores. El problema que tiene es que, al utilizar una multiplicación de factores (sería mejor una suma ponderada) y no tener en cuenta el número de paquetes asignados por conexión, para distintas distribuciones de paquetes dará el mismo valor, por lo que tiene muchas mesetas y no es capaz de distinguir bien entre soluciones mejores o peores. Sería más adecuado incluir el número de paquetes por conexión en el heurístico de forma que el operador mover tuviera un efecto directo en el valor del heurístico. Y dado que la solución inicial o los operadores pueden salir del espacio de soluciones, el heurístico debería penalizar las no-soluciones (de forma proporcional al número de paquetes asignados por encima de la capacidad de las conexiones).

- b) Se plantea solucionarlo mediante Hill-Climbing, tomando como solución inicial el repartir los paquetes entre las conexiones de salida del equipo asignando aleatoriamente a una conexión el máximo de su capacidad, repitiendo el proceso hasta haber repartido todos los paquetes. Como operador se utilizaría mover cualquiera de los paquetes de una conexión a otra siempre que la operación sea válida. Como función heurística usamos el sumatorio para todas las conexiones con paquetes asignados del producto entre el número de paquetes asignados a la conexión y el retraso de la conexión más el sumatorio, para todas las conexiones con paquetes asignados, del producto entre el número de paquetes asignados a la conexión y el número de saltos medio de esa conexión.

De nuevo Hill-Climbing es una elección adecuada para este problema.

La solución inicial es correcta (es una asignación completa de paquetes a conexiones que no excede la capacidad de ninguna conexión), su coste en el caso peor es $O(N)$ (hace falta llegar a la última conexión para asignar todos los paquetes) pero no tenemos ninguna garantía sobre su calidad porque las conexiones se escogen de forma aleatoria y no se tiene en cuenta ni el retraso de las conexiones escogidas ni el número medio de saltos de esas conexiones.

El operador es válido, cubre todo el espacio de soluciones (moviendo paquetes de uno en uno de una conexión a otra, podemos llegar a todas las posibles asignaciones de paquetes a conexiones sin superar la capacidad de ninguna conexión). En este caso, si la solución inicial es solución, este operador nos mantiene dentro del espacio de soluciones. Teniendo en cuenta que todos los paquetes de una conexión son iguales (tienen el mismo tamaño y no distinguimos entre ellos) solo tiene sentido hacer la operación una vez por conexión. Su factor de ramificación es $O(N^2)$ (esta vez solo se ha de escoger, para cada conexión de origen, una conexión de destino). Al ser un operador de granularidad tan pequeña hará también que la búsqueda sea mucho más larga. Y al igual que en el apartado anterior, no tiene sentido añadir un operador de intercambio o swap.

La función heurística tiene en cuenta todos los elementos del problema. Al tener en cuenta el número de paquetes asignados a cada conexión, permite diferenciar entre soluciones similares (reduce las mesetas) y hace que cada aplicación del operador mover modifique el valor del heurístico. Si la minimizamos obtendremos la solución que buscamos. No penaliza no-soluciones, pero en este caso no hace falta ya que no se generarán. El único defecto que tiene es que al estar mezclando dos criterios distintos las magnitudes que tienen pueden hacer que un criterio tenga preferencia respecto al otro a la hora de buscar una solución, y por ello lo mejor sería añadir un factor ponderante a cada sumando.

- c) Se plantea solucionarlo mediante Algoritmos Genéticos, donde la representación del problema es una tira de bits compuesta por la concatenación de la representación en binario del número de paquetes asignados a cada conexión (evidentemente usando el mismo número de bits para cada conexión). Como solución inicial ordenamos las conexiones por su retraso (de menor a mayor) y asignamos paquetes en

ese orden llenando la capacidad de cada conexión hasta haber asignado todos los paquetes. Usamos los operadores de cruce y mutación habituales. Usamos como función heurística el sumatorio, para todas las conexiones, del producto entre el número de paquetes asignados a cada conexión y el retraso de la conexión.

Algoritmos Genéticos es también una técnica adecuada al tener que obtener soluciones que maximicen o minimizan unos criterios sin requerir encontrar el óptimo global.

La codificación representa para cada conexión el número de paquetes asignados (en binario). No especifica exactamente el número máximo de paquetes que se intenta representar por conexión: en el caso peor ese número sería P , y por lo tanto la representación tendría tamaño $(N \times \lceil \log_2(P) \rceil)$ bits, pero lo que tendría más sentido sería limitar el número de paquetes por conexión a representar a la capacidad máxima entre las conexiones (supongamos que ese número es C_{max}), y en ese caso el tamaño de la representación se reduciría a $(N \times \lceil \log_2(C_{max}) \rceil)$. La representación propuesta permite representar correctamente todas las soluciones (asignaciones de números de paquetes a conexiones), pero también varios tipos de no-soluciones:

- NS1: si el número de bits asociado a cada conexión puede representar números por encima de la capacidad de la misma, podemos asignar paquetes por encima de la capacidad de esa conexión.
- NS2: asignaciones de menos de P paquetes (de hecho podemos asignar 0 paquetes con esta representación)
- NS3: asignaciones de más de P paquetes

Difícilmente se puede encontrar una codificación mejor que reduzca la cantidad de no-soluciones que pueda representar. Si giramos la representación -para cada paquete asignamos el número de conexión en binario- esta representación tendría tamaño $(P \times \lceil \log_2(N) \rceil)$ (si asumimos que $P > N$ el logaritmo está aplicándose a N que es el número menor, por lo tanto va a ocupar más bits que la representación propuesta en el enunciado). Y en esta representación, aunque evitamos NS2 y NS3, seguiríamos teniendo una variación de NS1 (podemos asignar a una conexión paquetes por encima de su capacidad) y se añade un nuevo tipo de no-solución (identificadores de conexión inválidos, si N no es una potencia de 2).

La solución inicial es solución (asignamos todos los paquetes, y lo hacemos sin superar la capacidad de ninguna conexión), su coste es $O(N \times \log(N) + N)$ (coste de la ordenación de las N conexiones más el de asignar números de paquetes a esas N conexiones). Este coste no será demasiado grande asumiendo que el número de conexiones no es demasiado alto. Esta solución intenta optimizar uno de los criterios (el retraso) esto no significa que la solución inicial sea buena respecto al objetivo del problema: al no tener en cuenta el otro criterio (el número de saltos) no tenemos garantías de estar cerca de las soluciones mejores aunque probablemente sea mejor que las alternativas de los otros apartados. El problema es que con este generador solo podemos generar una solución, y en Algoritmos Genéticos necesitamos generadores que nos aporten variabilidad para crear una población diversa de soluciones iniciales. De hecho, en este caso, el generador de solución inicial del apartado b) sería mejor ya que tiene un coste menor y además incluye aleatoriedad que aporta una cierta variabilidad.

Los operadores de cruce y mutación pueden dar lugar a soluciones invalidas ya que operan directamente sobre la representación sin comprobar la capacidad de las conexiones, e incluso pueden alterar el número total de paquetes asignados, que debería ser siempre P . En concreto:

- el operador de cruce en un punto:
 - con una población de soluciones iniciales sin variabilidad, en las primera iteraciones no tendrá efecto alguno (el cruce de dos codificaciones idénticas da como resultadoe sas dos codificaciones), y no empezará a tener un efecto hasta que el operador de mutación haya generado cierta variabilidad en las poblaciones intermedias,
 - puede provocar NS2 y NS3: supongamos que en una de las soluciones hay más paquetes asignados a la izquierda del punto de cruce, y en la otra solución hay más paquetes asignados a la derecha del punto de cruce. Después del cruce una solución tendrá exceso de paquetes (NS3) y la otra tendrá falta de paquetes (NS2),
 - puede provocar NS1: si el punto de cruce está dentro de la codificación en binario del número de paquetes asignados a una conexión c_i en ambas soluciones, tras el cruce es probable que en una las soluciones la conexión c_i tenga asignado un número por encima de su capacidad máxima (NS1).

- el operador de mutación de un bit cualquiera:
 - mutación de un bit de 1 a 0: esto reduce el número de paquetes asignados a alguna conexión sin colocarlos en otra, lo que provoca soluciones con menos de P paquetes (NS2):
 - mutación de un bit de 0 a 1: esto aumenta el número de paquetes asignados a alguna conexión sin sacarlos de otra, lo que provoca soluciones con más de P paquetes (NS3) e incluso puede superar la capacidad de esa conexión (NS1)

La función heurística es incompleta ya que solo tenemos en cuenta el retraso de las conexiones, pero no el número de saltos (de hecho, con este heurístico, la solución inicial planteada ya es la mejor solución). Esta función debería penalizar los diferentes tipos de no-soluciones (deberíamos tener un penalizador por cada tipo de no-solución, proporcional al desvío respecto a los criterios de ser solución). Y como el objetivo de los Algoritmos Genéticos es maximizar su función de fitness, en este caso (que queremos minimizarla) tenemos que cambiar el signo de la función.